

Десериализация объектов C++ из внешних источников данных

Зачастую при взаимодействии с внешними источниками данных (БД или сторонний сервис) приходится работать с форматом, который не соответствует системе типов Wasaby Framework. Поэтому между форматами требуется устанавливать соответствие с помощью некоторого программного слоя. В простейшем случае, в качестве такого слоя могут использоваться объекты классов [sbis::Record](#) или [sbis::RecordSet](#). При этом, если после десериализации необходимо дополнительно создать объект собственного типа, разработчику потребуется реализовать собственный код конвертации `sbis::Record` или `sbis::RecordSet` в требуемый тип.

Описанный подход обладает следующими недостатками:

- в памяти сервера создаётся и хранится временный объект класса `sbis::Record` или `sbis::RecordSet`;
- становится громоздким исходный код для создания типизированного объекта.

Пример неэффективного создания типизированного объекта C++.

```
1 // Выполняем запрос к БД.
2 // Получаем временный объект rs.
3 // Он хранится в памяти сервера и обладает всеми изменяемыми данными.
4 auto rs = sbis::SqlQuery( conn.mDB, L"SELECT * FROM profile WHERE
  id=$1"_sv, 42 );
5 ...
6
7 // Выполняются преобразования для создания типизированного объекта.
8 // Код преобразования получается достаточно громоздким.
9 auto profile = std::make_shared< ProfileModel >();
10
11 if( rec.TestFieldNotNull( L"id"_sv ) )
12     profile->uuid = GetSafeUuid(rec, L"id"_sv);
13
14 if( rec.TestFieldNotNull( L"timestamp"_sv ) )
15     profile->timestamp = profile->full_timestamp = rec[ L"timestamp"_sv
16     ].To< int64_t >();
17 ...
```

Для решения описанных проблем предлагается использовать пару интерфейсов [sbis::IRecordView](#) и [sbis::IFieldView](#), с помощью которых возможно чтение данные из источника сразу в типизированный C++ объект без накладных расходов.

Интерфейсы `sbis::IRecordView` и `sbis::IFieldView`

Легковесный интерфейс `sbis::IRecordView` предоставляет доступ к иммутабельным (только чтение; без редактирования) данным источника. Интерфейс является контейнером для `sbis::IFieldView`, который позволяет читать значения полей и конвертировать их в указанный тип данных. Конвертация возможна в следующие типы данных:

- Bool
- Int8, Int16, Int32, Int64, Float, Double, Decimal
- Date, Time, DateTime, TimeInterval
- String, Uuid
- Binary
- AnyValue
- ArrayBool, ArrayInt16, ArrayInt32, ArrayInt64, ArrayFloat, ArrayDouble, ArrayString, ArrayDate, ArrayTime, ArrayDateTime, ArrayTimeInterval, ArrayUuid, ArrayDecimal

Также в Wasaby Framework существует класс [sbis::RecordReader](#) — это основной шаблонный класс для реализации логики десериализации экземпляров IRecordView в произвольную структуру типа T.

Для того чтобы ваш класс можно было десериализовать в типизированный C++ объект, необходимо реализовать собственную специализацию шаблона. Это подробно рассмотрено в следующем подразделе.



Специализация шаблонного класса sbis::RecordReader должна заполнять все поля экземпляра типизированного объекта, подаваемого на вход методу Read. Если реализуется условное вычитывание поля из sbis::IRecordView, при невыполнении условия ему должно быть присвоено значение по умолчанию, либо иное значение, определяемое логикой чтения типизированного объекта.



Важно!

Незаполнение полей экземпляра типизированного объекта приводит к неопределённому поведению.

Десериализация в типизированный C++ с базовым набором типов полей. Использование шаблонного класса sbis::RecordReader

Приведённый ниже код можно генерировать автоматически из djinni-моделей, в описании которых применён атрибут [serializable](#).

Пример эффективного создания типизированного объекта C++.

```
1 // Объявляем собственную структуру данных, которую требуется получить
  от источника данных.
2 struct User
3 {
4     Int32 mAge;
5     String mName;
6     bool mGiveFreeBeer;
7 };
8
9 // Специализация класса для чтения объектов по структуре User.
10 template<>
11 struct RecordReader< User >
12 {
13     static void Read( User& data, IRecordView const& record )
14     {
15
16         // Из объекта типа sbis::IRecordView читаем поле с именем "age" и
        помещаем в поле mAge
17         ReadFieldFromRecord( data.mAge, record, L"age"_sv );
18
19         // Из объекта типа sbis::IRecordView читаем поле с именем "name"
        и помещаем в поле mName.
20         ReadFieldFromRecord( data.mName, record, L"name"_sv );
21
22         // Реализация сложной логики заполнения поля mGiveFreeBeer.
        Почему-то
23         // безусловно выставляем его в true для тех объектов, у которых
        mAge > 20 и mName = John,
24         // а иначе читаем из соответствующего поля
25         if( data.mAge > 20 && data.mName == L"John"_sv )
26             data.mGiveFreeBeer = true;
27         else
```

```

28         ReadFieldFromRecord( data.mGiveFreeBeer, record,
29         L"free_beer"_sv );
30     };

```

Десериализация в типизированный C++ с собственным типом полей. Использование шаблонных классов `sbis::RecordReader` и `sbis::FieldReader`

Если типизированный объект содержит поля, тип которых не соответствует базовым типам из списка выше, то вы можете реализовать собственную специализацию шаблонного класса [sbis::FieldReader](#) для чтения полей иных типов.

Пример эффективного создания типизированного объекта C++ с собственным типом поля.

```

1  // Прикладной перечислимый тип.
2  enum class Sex : Int32
3  {
4      MALE = 0,
5      FEMALE = 1
6  };
7
8  // Модель данных.
9  struct User
10 {
11     ...
12     // Поле прикладного типа Sex, для которого платформа не предоставляет
    специализацию шаблона FieldReader.
13     Sex mSex;
14     ...
15 };
16
17 // Специализация шаблонного класса для чтения объектов перечислимого
    типа Sex.
18 template<>
19 struct FieldReader< Sex >
20 {
21     static void Read( Sex& data, IFieldView const& field )
22     {
23
24         // Получаем значение переданного поля, преобразуем к типу Int32 и
    возвращаем в качестве значения объекта типа Sex.
25         data = static_cast< Sex >( field.GetValue< Int32 >() );
26     }
27 };
28
29 // Структура типизированного объекта C++ будет создана по объекту типа
    User.
30 template<>
31 struct RecordReader< User >
32 {
33     static void Read( User& user, IRecordView const& rec )
34     {
35         ...

```

```

36         // В следующей строке из объекта типа sbis::IRecordView
        вычитывается поле "sex" и помещается в поле mSex структуры User при
        помощи
37         // вышеописанной специализации FieldReader< Sex >. Если данная
        специализация не будет видна в точке определения
38         // специализации RecordReader'a, то эта строка не скомпилируется.
39         ReadFieldFromRecord( user.mSex, rec, L"sex"_sv );
40         ...
41     }
42 };

```

Новые методы для чтения типизированных объектов C++ из БД

В комбинации с описанными выше интерфейсами в сервисном фреймворке реализованы новые методы, которые позволяют вычитывать данные из БД сразу в типизованный объект C++:

- `sbis::dbi::IStatement::Exec`
- `sbis::SqlQuery`
- `sbis::dbi::SqlQueryOf`

`sbis::dbi::IStatement::Exec`

Возвращает объект типа `sbis::StructCursor`, который итерирует результат запроса по `sbis::IRecordView` в БД и выдаёт их в виде типизированных структур.

```

1  auto statement = dbi::GetConnectedDataDB()->CreateStatement();
2  auto cursor = statement->Exec< Profile >( L"SELECT * FROM profile WHERE
   id=$1"_sv, 42 );
3  while( cursor.MoveNext() )
4  {
5      // Возвращает типизированную модель.
6      Profile const& p = cursor.GetCurrent();
7
8      // Работа с объектом p.
9      ...
10 }

```

`sbis::SqlQuery`

Возвращает объект типа `sbis::StructCursor`, который итерирует результат запроса по `sbis::IRecordView` в БД и выдаёт их в виде типизированных структур.

Сигнатура:

```
SqlQuery < тип объекта > ( соединение, строка запроса, параметры );
```

где

- **тип объекта** — типизированный объект, который нужно получить в результате обработки одной записи `recordset'a`.
- **соединение** — объект класса [sbis::dbi::IDatabase](#).
- **строка запроса** — SQL-инструкция в строковом представлении.
- **параметры** — параметры, передаваемые в строку SQL-запроса.

Пример

```
1  auto cursor = SqlQuery<Profile>( conn.mDB, L"SELECT * FROM profile
```

```

1 WHERE id=$1"_sv, 42 );
2 while (cursor.MoveNext())
3 {
4     // Возвращает типизированную модель.
5     Profile const& p = cursor.GetCurrent();
6
7     // Работа с объектом p.
8     ...
9 }

```

sbis::dbi::SqlQueryOf

Метод возвращает контейнер с типизированными объектами.

Синтаксис вызова:

```
SqlQueryOf< тип контейнера<тип объекта> >( соединение, строка запроса,
параметры );
```

где

- **тип контейнера** — тип возвращаемого контейнера. Можно выбрать одно из значений: `std::vector`, `std::set`, `std::deque`, `std::list`.
- **тип объекта** — типизированный объект, который нужно получить в результате обработки одной записи `recordset'a`.
- **соединение** — объект класса [sbis::dbi::Database](#).
- **строка запроса** — SQL-инструкция в строковом представлении.
- **параметры** — параметры, передаваемые в строку SQL-запроса.

Пример

```

1 std::vector< Profile > profiles = SqlQueryOf< std::vector< Profile > >(
2     conn.mDB, L"SELECT * FROM profile WHERE id=$1"_sv, 42 );
3
4 for( Profile const& p: profiles)
5 {
6     // Работа с объектом p.
7 }

```

Интеграция с PyDjinni

При использовании djinni-моделей для интеграции с вышеописанным механизмом можно воспользоваться атрибутом **serializable**. В этом случае специализация [sbis::RecordReader](#) сгенерируется автоматически в соответствии с данными [аннотациями](#). Подробнее читайте в [статье](#).

Например, из описания djinni-модели:

```

1 # Задаём обязательный атрибут генерации для djinni-модели.
2 # Тип объекта с именем Profile.
3 profile = record [serializable] {
4
5     # Имя поля записи (в БД) и имя поля модели (для типизированного C++
6     # объекта) не отличаются.
7     name: string;
8     id: uuid;
9     phone: string;
10 }

```

Будет автоматически сгенерирована специализация:

```
1 struct Profile
2 {
3     String mName;
4     Uuid mId;
5     String mPhone;
6 };
7
8 // Специализация класса для чтения объектов по структуре Profile.
9 template<>
10 struct RecordReader< Profile >
11 {
12     static void Read( Profile& data, IRecordView const& record )
13     {
14         ReadFieldFromRecord( data.mName, record, L"name"_sv );
15         ReadFieldFromRecord( data.mId, record, L"id"_sv );
16         ReadFieldFromRecord( data.mPhone, record, L"phone"_sv );
17     }
18 };
```

В результате чего при выполнении запроса можно использовать конструкцию **SqlQueryOf** (см. [Выше](#)).